
Chapter 4

The Credit Game: Machine Learning and Explainability

“One problem in the field of statistics has been that everyone wants to be a theorist. Part of this is envy—the real sciences are based on mathematical theory. In the universities for this century, the glamor and prestige has been in mathematical models and theorems, no matter how irrelevant.”

— Leo Breiman, “Reflections After Refereeing Papers for NIPS,” 1995 [7]

By the 1990s, computing power had grown exponentially while statistics departments remained anchored to methods designed for the pre-computer era. As processors became faster and data became abundant, new disciplines emerged to exploit these resources: pattern recognition in computer science and engineering, neural networks spanning psychology and computer science (CS), machine learning in artificial intelligence (AI) labs. These fields shared a defining characteristic: computation, a fundamental element of their methodology. Yet, computation did not become a fundamental element in statistics.

The roots of this divergence stretched back decades. In the 1950s, D.J. Finney had written to R.A. Fisher about computing’s role [1]. Fisher viewed computers as tools to verify approximations, not substitutes for scientific thinking. He feared that scientists who handed data to machines risked becoming servants of computation rather than interpreters of their hypotheses. By 2001, Fisher’s fears had materialized, but not in the way he imagined. The problem was not that statisticians had embraced computation too eagerly; rather, it was because they had embraced it too timidly.

Jerome Friedman (who had never formally studied statistics) articulated this failure in his 2001 lecture “The Role of Statistics in the Data Revolution” [19]. Statistics, he argued, was “no longer the only game in town.” The discipline had defined itself by a fixed curriculum, those methods taught in graduate programs, while systematically excluding computing methodology. “Computing has been one of the most glaring omissions,” Friedman observed. “Had we incorporated computing

methodology from its inception as a fundamental statistical tool (as opposed to simply a convenient way to apply our existing tools) many of the other data related fields would not have needed to exist. They would have been part of our field.”

Leo Breiman, Friedman’s colleague at Stanford, was even more vocal. In his 2002 comment on statistical fraud detection [2], he observed: “We are ceding some of the most interesting of current statistical problems to computer scientists and engineers allied to the machine learning area.” His 2001 paper “Statistical Modeling: The Two Cultures” formalized this critique, arguing that 98% of statisticians focused on elegant parametric models (logistic regression, Cox models) that prioritized interpretability and inference, while the remaining 2%—mostly computer scientists—built algorithmic models (neural networks, random forests) that prioritized predictive accuracy. As Breiman described it, “The goal is predictive accuracy and the tools are algorithmic models. The class of problems is novel, even in machine learning. No one tool (neural nets, etc.) is instantly applicable to all of these problems. The algorithms have to be designed to fit the data. This means that an essential part of the venture is immersion in and exploration of the data.”

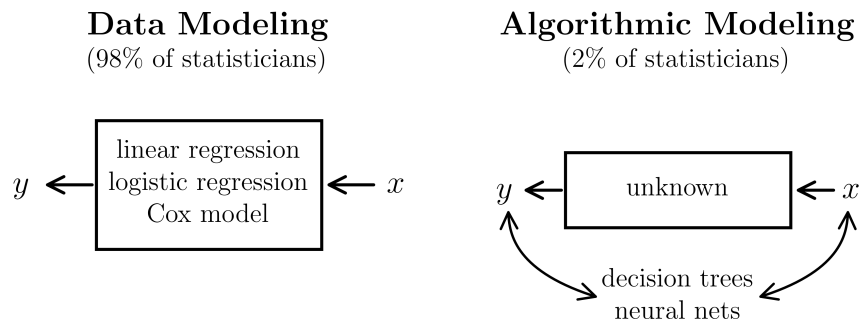


Figure 4.1: Breiman’s two cultures of statistical modeling: data modeling (interpretable, parametric) versus algorithmic modeling (predictive, black-box). Adapted from [8].

Breiman was an unorthodox statistician. In 1967, he left academia to become a full-time consultant [31]—a rare move for a tenured professor. As Adele Cutler, his collaborator on Random Forests¹, observed, “By and large, statisticians are not great risk-takers. We tend not to stray too far from what we know, tend not to tackle problems for which we have no tools, tend to adopt or adapt existing ideas

¹While Breiman published the seminal Random Forests paper alone in 2001 [6], Cutler was instrumental in developing the original Fortran and C implementations of the algorithm, helping to make Random Forests one of the most widely used machine learning methods.

instead of coming up with completely new ones” [12]. Breiman was the exception. His consulting work immersed him in real-world problems beyond the reach of traditional statistics.

In 1976, Breiman met Jerome Friedman, a high-energy particle physicist who had also moved into consulting. The two shared more than professional interests. Both had stepped outside traditional academic paths, both prioritized solving real problems over theory, and both had grown frustrated with statistics’ resistance to change. Soon Friedman was working alongside Breiman, and together with Richard Olshen and Charles Stone, they began collaborating on classification trees. The four decided to publish their research as a book rather than journal articles, apparently believing the work was unlikely to be accepted by standard statistical journals [31].

That book, *Classification and Regression Trees* (CART), became one of the most influential works in classical machine learning. United by their frustration with the statistical establishment, Breiman and Friedman would spend the next three decades building the algorithmic culture that statistics had rejected. Their work on classification trees [4], bagging [5], Random Forests [6], and gradient boosting demonstrated that statistical rigor and computational flexibility could coexist.

Even as collaborators, Breiman and Friedman disagreed on fundamental questions. When Friedman and his colleagues published their LogitBoost paper [17], seeking to explain boosting through maximum likelihood, Breiman remained unconvinced. As Friedman later recalled, they had figured it out “at least to our satisfaction... but not to everyone’s satisfaction. Leo never thought our explanation was the essential reason.”

Breiman saw it differently. “The major point of this comparison is to show that correlation and strength are the essential ingredients of an accurate ensemble, and that if boosting succeeds, it is because it gives low correlation between classifiers of reasonable strength. The fact that a similar algorithm maximizes the likelihood in a logistic setting is not the primary reason for its success.” For Breiman, the key was diversity among weak learners, not statistical optimality. Friedman saw boosting through the lens of maximum likelihood estimation (maximum likelihood, Fisher’s elegant framework from Chapter 2, had “perked [his] interest in statistics” [13]). Breiman saw it through the lens of ensemble diversity and variance reduction.

History sided with both. Gradient boosting machines (GBM), built on Friedman’s framework of gradient-boosted regression trees, took the crown as the dominant method for tree ensembles and powers countless production systems today. Yet Breiman’s insight about correlation and diversity proved equally essential, particularly for Random Forests, which continue to be a Swiss army knife for many data science problems.

Yet the better these algorithms performed, the harder they became to explain. The solution came from an unexpected direction: game theory. Scott Lundberg and Su-In Lee’s TreeSHAP algorithm [25, 22] provided exact, efficient computation of Shapley values for tree ensembles, connecting Lloyd Shapley’s 1950s framework [30] to modern machine learning. This breakthrough enabled tree ensembles to be deployed in high-stakes domains like credit risk. This chapter explores both dimensions: the predictive power of ensemble methods and the game-theoretic foundations that made them explainable.

4.1 From Statistical to Algorithmic Modeling

In statistics, *regression* refers to modeling the conditional expectation of a target variable, continuous or discrete. Logistic regression, covered extensively in Chapter 2, is fundamentally a regression algorithm: a Generalized Linear Model (GLM) that models the log-odds of a binary outcome. Developed over several decades (1930s–1970s by Berkson, McFadden [26], Cox [11], Nelder, and Wedderburn [29]), it was designed for experimental research analyzing treatment effects, testing hypotheses, and studying risk factors.

In machine learning, *regression* typically refers to predicting continuous outcomes, while *classification* refers to discrete labels. Under this terminology, logistic regression becomes a “classifier,” though statistically it remains a regression model. Throughout this chapter, we adopt the machine learning perspective and view classification as what Ronald Fisher called “a problem of probability” [14].

Early tree methods appeared in social science research during the 1960s and 1970s, but Breiman, Friedman, Olshen, and Stone’s CART (Classification and Regression Trees) became the definitive framework. Published in 1984 [10], CART formalized the recursive partitioning approach that became a cornerstone of modern data science, though alternative algorithms like CHAID, ID3, and C4.5 developed along parallel lines.

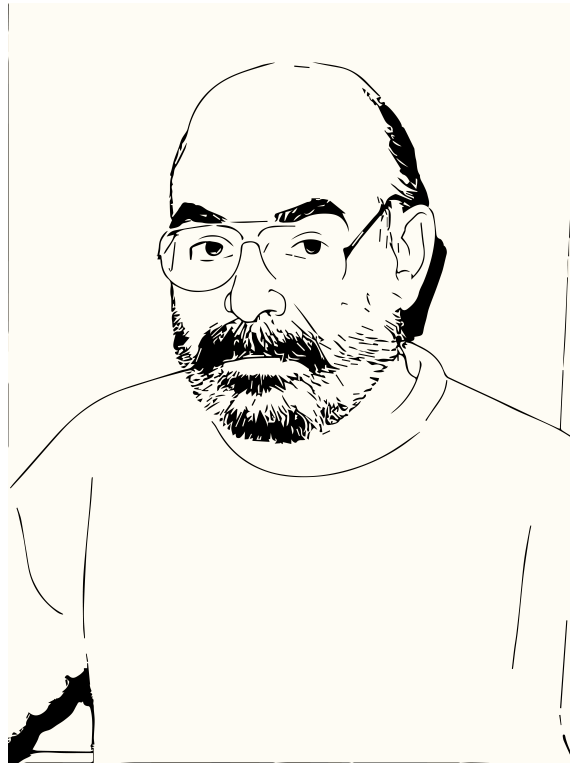


Figure 4.2: Leo Breiman (1928-2005), statistician and pioneer of classical machine learning. Creator of random forests and leading author of CART (Classification and Regression Trees), Breiman challenged the statistical establishment with his 2001 paper “Statistical Modeling: The Two Cultures,” arguing that the field’s fixation on interpretable models came at the cost of predictive accuracy.

What is a Decision Tree?

CART constructs trees through a process called recursive binary splitting. At each node, the algorithm asks a simple yes/no question about a single feature: Is applicant income $\leq \$5000$? or does she have a credit history? The data flows down the tree based on these answers until reaching a leaf node that provides the final prediction.

The use of classification trees arose for problems that could not be handled in an easy or natural way by traditional methods. Consider a loan approval problem with risk drivers like applicant income, co-applicant income, credit history, and requested loan amount. A logistic regression would model these as additive effects with explicit interaction terms. A decision tree discovers these patterns automatically.

Decision Tree: A Non-Parametric Learner

Decision trees are *non-parametric*: they make no assumptions about the functional form relating features to outcomes. Unlike logistic regression’s fixed linear structure, a tree’s complexity—its depth, number of splits, and partition boundaries—adapts to the data.

At each node, CART evaluates every possible threshold for every available feature to find the split that best separates the classes. The quality of a split is measured by an **impurity metric**, which quantifies how mixed the classes are within a set of samples.

Impurity Measures

Two quality metrics are commonly used to evaluate splits in CART decision trees: Gini impurity and entropy.

Gini impurity measures the probability of misclassifying a randomly chosen element:

$$\text{Gini}(S) = 1 - \sum_{k=1}^K p_k^2 \quad (4.1)$$

Entropy measures the information content or uncertainty in the distribution:

$$\text{Entropy}(S) = - \sum_{k=1}^K p_k \log_2(p_k) \quad (4.2)$$

Split Evaluation

To evaluate a candidate split, we first calculate the **parent impurity**, the impurity of all samples at the current node before any split is made. For a candidate split that divides the current node S into left child S_L and right child S_R , the **information gain** measures the reduction in impurity:

$$\text{Gain} = \text{Impurity}(S) - \left(\frac{|S_L|}{|S|} \text{Impurity}(S_L) + \frac{|S_R|}{|S|} \text{Impurity}(S_R) \right) \quad (4.3)$$

Here, $\text{Impurity}(S)$ is the parent impurity (either Gini or Entropy), and the weighted term $\frac{|S_L|}{|S|} \text{Impurity}(S_L) + \frac{|S_R|}{|S|} \text{Impurity}(S_R)$ represents the expected impurity after the split. The algorithm selects the split with maximum gain, then recursively applies this process to each child node until reaching a stopping criterion such as maximum depth, minimum samples per node, or nodes that are sufficiently pure.

A Note on Credit Risk Terminology

Credit risk practitioners typically work with **Information Value (IV)** and **Weight of Evidence (WOE)** rather than Gini or entropy. As discussed in Chapter 1, these concepts are deeply connected through information theory. Information gain from a split measures how much the data reduce our uncertainty about the outcome (the weight of evidence) in favor of that split. The connection between decision tree splitting criteria and the log-likelihood ratios familiar in credit risk modeling is explored in detail below.

Information Gain as an Increase in Plausibility

To select the most optimal splits, we use a gain metric that measures how much uncertainty decreases after conditioning on a feature. Formally, it equals the mutual information $I(X; Y)$ between the feature and the target variable, measuring the reduction in uncertainty:

$$I(X; Y) = H(Y) - H(Y|X) \quad (4.4)$$

This is equivalent to the expected Kullback-Leibler (KL) divergence between the conditional and marginal class distributions:

$$I(X; Y) = \sum_x P(x) KL(P(Y|X = x) || P(Y)) \quad (4.5)$$

Information gain in decision trees plays the same conceptual role that Weight of Evidence plays in scorecards: both quantify the plausibility gained from observing new evidence. Like WOE, gain measures how much more predictable the outcome becomes after conditioning on a feature. However, there are two key differences:

- **No contrasting hypotheses:** While WOE compares the log odds of default versus non-default (a log-likelihood ratio), information gain measures reduction in uncertainty about a single distribution, how much more concentrated the class probabilities become after the split.
- **Local evaluation:** Trees apply this logic *locally* at each partition. Each split evaluates gain conditionally within its local region (the current node), not globally across the entire dataset. A feature might provide high gain at the root but no additional gain deeper in the tree, once other features have already partitioned the space. This recursive, local evaluation is what allows trees to capture complex interactions.

More precisely, gain measures how strongly a feature splits a local partition into child groups whose class distributions differ from the parent. Each split thus

measures how much more plausible the correct outcome becomes after observing the feature *within that specific region of feature space*.

Entropy, one measure of impurity, at each node is calculated using the maximum likelihood estimate, the empirical frequencies within that node's sample. If a node contains n_1 defaults and n_0 good customers, the entropy is $H = -\hat{p} \log_2(\hat{p}) - (1 - \hat{p}) \log_2(1 - \hat{p})$ where $\hat{p} = n_1/(n_1 + n_0)$ is the MLE. Information gain is simply the difference between the parent's entropy and the weighted average of the children's entropies, each computed from their local MLEs.

Figure 4.3 illustrates how information gain quantifies the plausibility gained from a split. The parent node contains a mixed population of 30 instances (14 defaults shown as gray squares, 16 good customers shown as white circles). When we split on $\text{Income} \leq \$50\text{k}$, the left child becomes concentrated with defaults (13 of 17), while the right child becomes concentrated with good customers (12 of 13). The information gain of 0.381 measures the uncertainty reduction after the split.

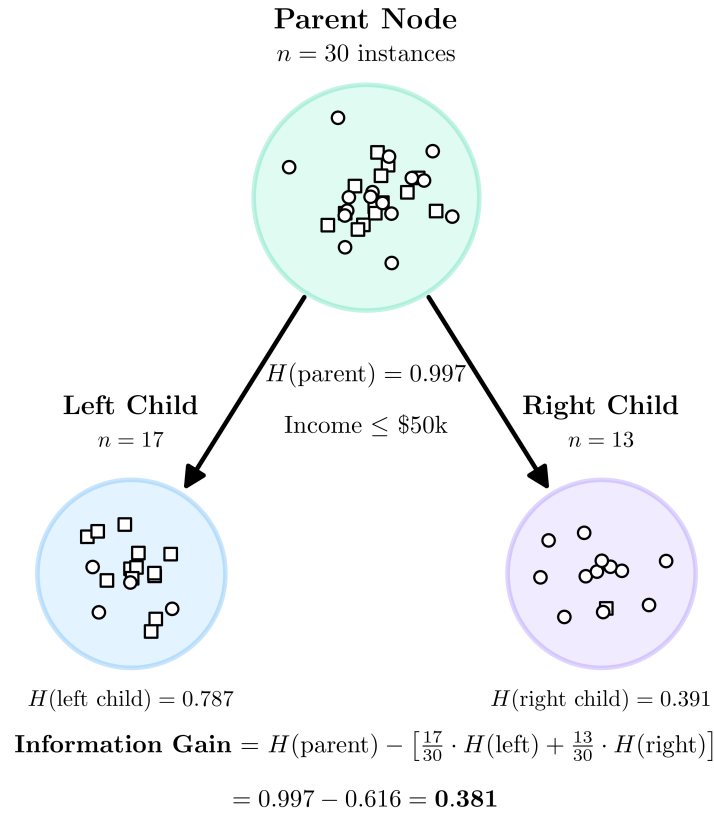


Figure 4.3: Information gain from a binary decision tree split.

When using entropy as the impurity measure, calculations at each node rely on maximum likelihood estimates—the empirical frequencies within that node's sample. If a node contains n_1 defaults and n_0 non-defaults, the entropy is $H =$

$-\hat{p} \log_2(\hat{p}) - (1 - \hat{p}) \log_2(1 - \hat{p})$ where $\hat{p} = n_1/(n_1 + n_0)$ is the maximum likelihood estimate (MLE). Information gain is the difference between the parent's entropy and the weighted average of the children's entropies, each computed from their local MLEs.

Split Selection in Practice

To see how this works in practice, let's build a tree to predict loan approval using only the Income feature. Figure 4.4 shows an example of the split selection process.

At the root node, we have all 9,721 loan applications. The algorithm evaluates every possible income threshold and selects $s_1 = 3813$ because it maximizes information gain in the full data distribution. The root split represents the best available condition to separate data points.

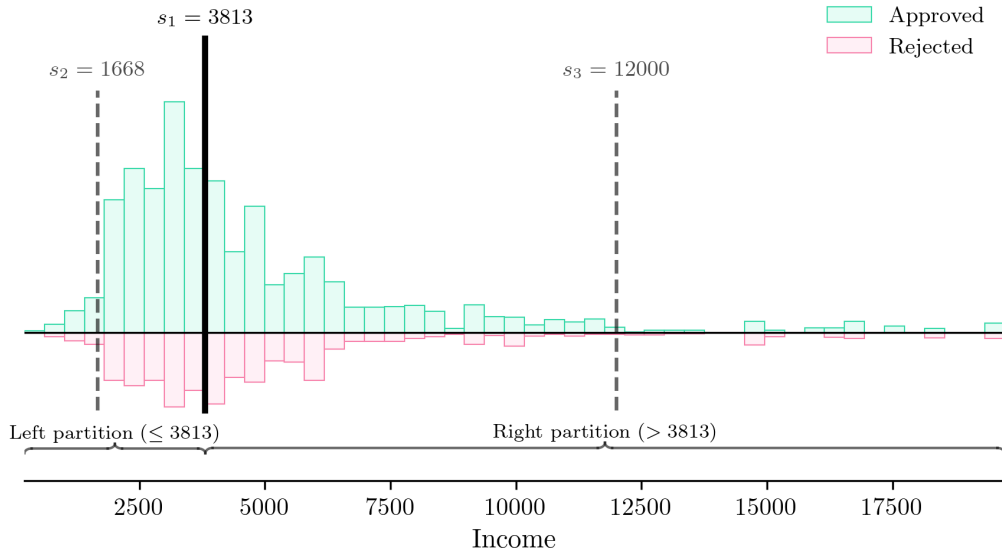


Figure 4.4: Split selection process for loan approval based on Income. The histogram shows approved loans (light gray, above axis) and rejected loans (dark gray, below axis). The vertical lines mark splits chosen by the algorithm at different depths of the tree.

Crucially, **the parent entropy is recalculated at each level**. After the root split:

- The **left partition** (Income ≤ 3813) contains 4,719 samples. Within that partition, the algorithm finds the best split at $s_2 = 1668$, indicating these child nodes are already quite pure.
- The **right partition** (Income > 3813) contains 5,002 samples. In this subregion, the algorithm splits at $s_3 = 12000$.

Each partition has its own baseline impurity, and gains are always calculated relative to the samples that actually reached that node. The resulting tree structure is shown in Figure 4.5.

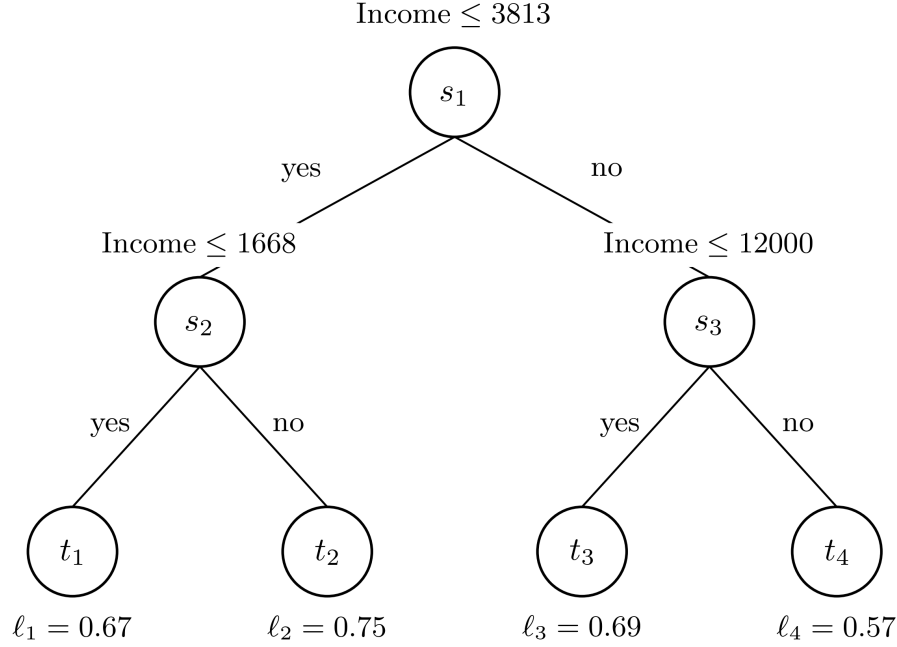


Figure 4.5: Decision tree built from Income alone. The tree recursively partitions applicants into increasingly homogeneous groups. Each terminal node t_i shows the predicted probability ℓ_i of loan approval for applicants in that partition.

Capturing Feature Interactions

Now, assume that we wanted to add an additional risk driver, co-applicant income. How should the tree decide which feature to split on at each node?

The algorithm chooses the split that best separates approved from rejected applications. In this case, the tree captures the interaction between applicant income and co-applicant income without explicitly specifying it: an applicant with low primary income but high co-applicant income can still qualify, because the tree will route it through different branches based on the co-applicant's contribution.

With two features, the tree structure becomes richer: the root node might split on co-applicant income first (if that provides the best separation), then subsequent nodes split on applicant income. The order is determined entirely by the data.

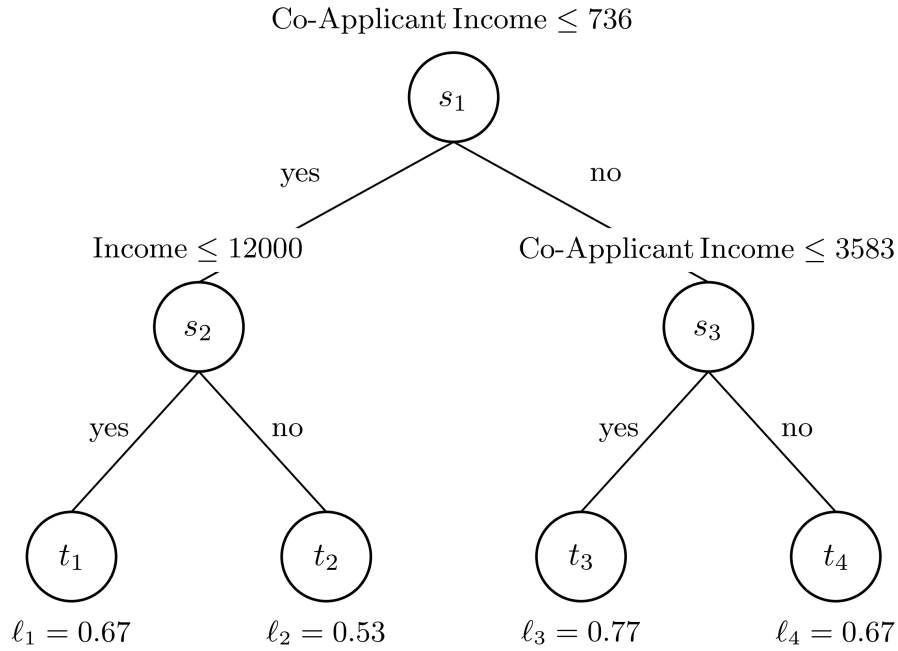


Figure 4.6: Decision tree using two features (Income and Co-applicant Income). Introducing an additional predictor creates new branches and allows the tree to capture interactions between income sources.

The visualization in Figure 4.7 reveals the geometric interpretation of decision trees: each split creates an axis-aligned partition of the feature space. The tree cannot create diagonal boundaries², it can only split horizontally or vertically. This is both a strength (interpretability) and a limitation (inefficiency for diagonal patterns). Without regularization constraints such as maximum depth or minimum samples per leaf, trees will continue splitting until each leaf contains a single training example, perfectly memorizing the training data but generalizing poorly to new observations.

²Trees have three key properties: (1) No extrapolation beyond training ranges—all out-of-range values follow the same path. (2) Invariance to monotonic transformations—only rank ordering matters, not scale. (3) Histogram optimization—modern implementations pre-bin features (256 buckets) rather than evaluate every split, mirroring the binning credit risk modelers do manually.

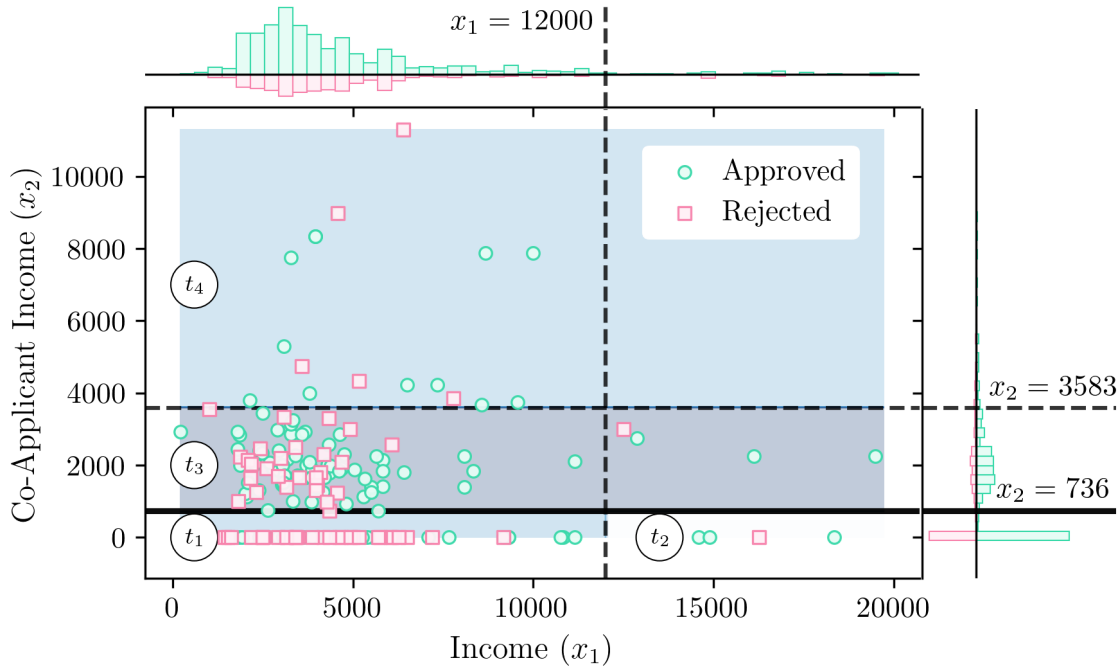


Figure 4.7: Decision boundary for a tree with two features. Each axis-aligned line represents a split, creating rectangular regions. The color intensity shows the predicted approval probability in each region. Notice how the tree creates a step-function approximation to the true decision boundary.

Decision Trees in Credit Risk Practice

In credit risk modeling, decision trees play a specialized role beyond prediction: they provide optimal binning for Weight of Evidence (WOE) transformations discussed in Chapter 2. The tree's splits naturally identify breakpoints where the relationship between a predictor and default risk changes, creating bins that maximize information value.

Credit risk modeling standards impose additional constraints. Regulatory and business requirements typically mandate that each leaf (bin) contains at least 5% of observations to ensure statistical stability and prevent overfitting to small subgroups. Additionally, monotonicity is often preferred: as a risk driver increases (e.g., debt-to-income ratio), the predicted default rate should move in a consistent direction. This ensures the scorecard aligns with economic intuition and regulatory expectations, even if it sacrifices some predictive accuracy.

Seeing the (Random) Forest for the Trees

Leo Breiman's Random Forest algorithm [6] introduced two sources of randomness to address a fundamental problem: single decision trees overfit to their training data. The first, **bagging** (bootstrap aggregating) [5], creates diversity by training each tree on a different bootstrap sample of the data, where each tree sees roughly two-thirds of the training examples [3]. However, bagging alone has a limitation: trees trained on bootstrap samples tend to be highly correlated [28], especially when a strong predictor exists, limiting the variance reduction that averaging can achieve.

Breiman's second innovation, **feature randomization**, addresses this by restricting each split to consider only a random subset of m features, typically $m = \sqrt{p}$ for classification, where p is the total number of features [3]. This decorrelates the trees by forcing the forest to explore different feature combinations, ensuring that even if one feature is strongest overall, it will be unavailable at many splits, allowing other features to contribute.

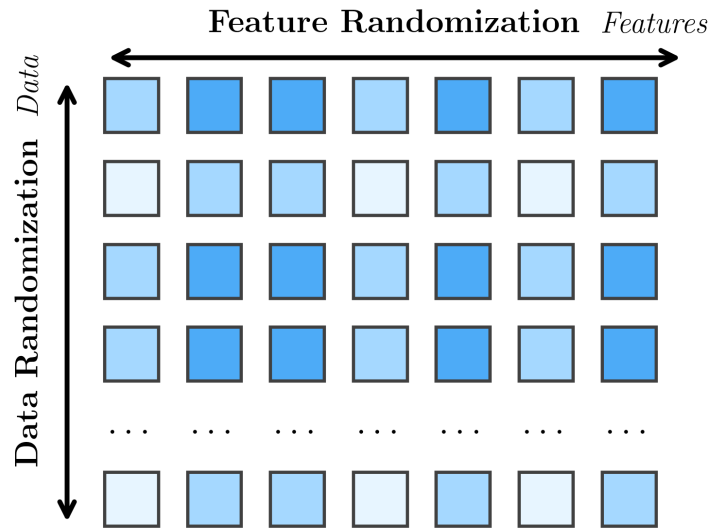


Figure 4.8: Two sources of randomness in Random Forests. Horizontal: The original dataset with n observations and p features. Vertical: Bagging creates a bootstrap sample containing roughly two-thirds of observations, with some repeated and others out-of-bag.

This combination addresses the fundamental bias-variance tradeoff in statistics and machine learning. Individual decision trees have low bias (they can fit complex patterns) but high variance (they overfit to training data). Random Forests maintain the low bias of trees while dramatically reducing variance through averaging. The feature randomization ensures that this averaging is effective by keeping the trees

sufficiently decorrelated, resulting in a collection of diverse trees that, when averaged, achieve lower prediction error than any individual tree.

Fisher-Yates Shuffle for Feature Randomization

Modern Random Forest implementations use the Fisher-Yates shuffle algorithm [15] to select features randomly without replacement. Originally developed by Ronald Fisher and Frank Yates for randomizing experimental designs [15], this algorithm samples m features from p available features through a partial shuffle: it iterates through positions, swapping each with a randomly chosen position, ensuring every possible subset has equal probability of selection.

Fisher-Yates Shuffle: Random Sampling of 3 Features from 7

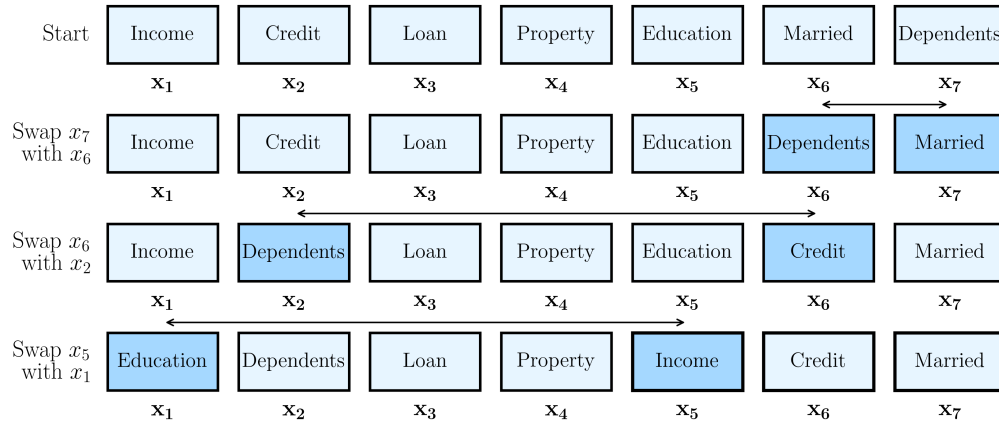


Figure 4.9: Fisher-Yates shuffle algorithm in action. Starting with an ordered array, the algorithm iterates from right to left, swapping each position i with a randomly chosen position $j \leq i$. Gray boxes indicate the positions being swapped. This ensures each permutation is equally likely, providing unbiased feature selection for Random Forests.

Sampling m features takes only $O(m)$ time, and every possible subset of m features has equal probability $\frac{1}{\binom{p}{m}}$ of being chosen. This unbiased randomization gives Random Forests their decorrelating power.

Aggregating Tree into a Forest

In classification tasks, each tree votes for a class. The forest combines these votes either through majority voting (hard predictions) or by averaging probabilities (soft predictions). Mathematically, if tree t predicts probability $\hat{p}_t(x)$ for class 1, the forest prediction is:

$$\hat{p}_{\text{forest}}(x) = \frac{1}{T} \sum_{t=1}^T \hat{p}_t(x) \quad (4.6)$$

This averaging reduces variance dramatically: even if individual trees overfit, their errors tend to cancel out when averaged. The forest prediction is the empirical mean of the tree predictions, estimating the expected prediction under the random forest data generation process.

Random Forest's Individual Tree Predictions

	p_1	p_2	p_3	p_4	p_5	p_6	p_7	$\dots$		Decision
$\mathbf{id}_i$	0.74	0.65	0.30	0.90	0.65	0.85	0.70	$\dots$	$\frac{1}{T} \sum p_j$ $= 0.68$	Yes
$\vdots$										
$\mathbf{id}_j$	0.05	0.03	0.10	0.01	0.02	0.08	0.01	$\dots$	$\frac{1}{T} \sum p_j$ $= 0.04$	No

Figure 4.10: Random Forest aggregates predictions from individual trees. Each tree provides an independent probability estimate based on its bootstrapped sample and random feature subset. The forest averages these predictions to produce the final output. Instance $\mathbf{id}_i$ receives high approval probability (0.68) with consistent tree agreement, while $\mathbf{id}_j$ shows strong rejection signal (0.04) across all trees.

Out-of-Bag Error Estimation and Uncertainty Quantification

Random Forests provide a unique advantage through out-of-bag (OOB) error estimation. Since each tree is trained on a bootstrap sample containing roughly two-thirds of the data, the remaining one-third serves as a natural test set for that tree. For each observation, we compute a prediction using only the trees that did not include it in their training sample, providing an unbiased estimate of the forest's generalization error without requiring a separate validation set. For credit risk applications, this means we can monitor model performance continuously as we add trees, stopping when OOB error stabilizes rather than relying on arbitrary choices of forest size.

The ensemble prediction $\hat{p}_{\text{forest}}(x)$ has a natural statistical interpretation. Since each tree t provides a binary vote (or probability), the forest prediction is the sample mean of T independent predictions from bootstrapped datasets. This connects to binomial inference: if we treat each tree's prediction as a Bernoulli trial, the variance of the forest prediction is:

$$\text{Var}[\hat{p}_{\text{forest}}(x)] = \frac{\hat{p}_{\text{forest}}(x)(1 - \hat{p}_{\text{forest}}(x))}{T} \quad (4.7)$$

This allows us to construct confidence intervals for predictions using a standard inference on binomial proportion (e.g., Wilson score interval or Clopper-Pearson). For a 95% confidence interval:

$$CI_{95\%} = \hat{p}_{\text{forest}}(x) \pm z_{0.975} \sqrt{\frac{\hat{p}_{\text{forest}}(x)(1 - \hat{p}_{\text{forest}}(x))}{T}} \quad (4.8)$$

where $z_{0.975} = 1.96$ for a 95% confidence level.

As the number of trees grows, the confidence interval width decreases proportionally to $\frac{1}{\sqrt{T}}$, providing increasingly precise estimates of the forest's prediction. This inverse relationship between ensemble size and prediction uncertainty explains why larger forests yield more reliable probability estimates with narrower confidence bounds.

Random Forests in Credit Risk

Random Forests offer particular value in credit risk modeling through their ability to detect variable interactions automatically. Credit risk drivers often interact in meaningful ways: affordability ratios like expense-to-income or loan-to-value are themselves interaction terms between income, expenses, and loan amounts.

Random Forests can guide the enhancement of traditional scorecards through a practical workflow: first, train a Random Forest and examine variable importance measures to identify which interactions contribute most to predictive power. Then, test these interactions in logistic regression models, validating performance out-of-sample to avoid overfitting.

Beyond prediction, Random Forests serve multiple roles in credit risk: trees can clean variables, identify optimal cutoff points, segment data, and generate candidate credit policies. Their ability to handle multicollinearity and provide variable importance rankings makes them useful for exploring feature interactions.

A Philosophical Note

RF is an example of a tool that is useful in doing analyses of scientific data. But the cleverest algorithms are no substitute for human intelligence and knowledge of the data in the problem.

Take the output of random forests not as absolute truth, but as smart computer generated guesses that may be helpful in leading to a deeper understanding of the problem.

— “Random Forests.” Leo Breiman and Adele Cutler [9].

Gradient Boosting: From AdaBoost to Modern Implementations

Boosting algorithms build strong predictive models by sequentially combining weak learners. Unlike Random Forests, which train trees independently and average their predictions, boosting trains each tree to correct the mistakes of its predecessors.

A Conceptual Note

Understanding what trees predict is essential for interpreting ensemble models:

- **Random Forests:** Each tree is a *classification tree* that predicts class probabilities (default rates). The forest averages these probabilities: $\hat{p}(x) = \frac{1}{T} \sum_{t=1}^T p_t(x)$, where each $p_t(x)$ is a probability estimate from tree t .
- **Gradient Boosting:** Each tree is a *regression tree* that predicts residuals in log-odds space. Trees do not output probabilities directly—they output corrections to the current log-odds: $F^{(m)}(x) = F^{(m-1)}(x) + \eta \cdot h_m(x)$, where $h_m(x)$ is the residual correction. The final probability is obtained by applying the logistic function: $\hat{p}(x) = \sigma(F^{(M)}(x))$.

Random Forest leaf values are probabilities, while gradient boosting leaf values are log-odds corrections. Both produce probabilities as final outputs, but through fundamentally different mechanisms. This parallel to logistic regression’s iterative optimization (Fisher scoring) will become clear below.

AdaBoost: Adaptive Boosting

AdaBoost [16] introduced the core boosting principle: iteratively reweight training examples to focus subsequent learners on previously misclassified instances. The algorithm maintains a weight $w_i^{(t)}$ for each training example, increasing weights for misclassified points and decreasing weights for correctly classified ones.

The AdaBoost update rule for binary classification is:

$$F^{(t)}(x) = F^{(t-1)}(x) + \alpha_t h_t(x) \quad (4.9)$$

where $h_t(x) \in \{-1, +1\}$ is a weak classifier and $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$ weights the classifier based on its error rate ϵ_t .

LogitBoost: Additive Logistic Regression

LogitBoost [17] adapts boosting to logistic regression by minimizing the binomial log-likelihood. The algorithm reveals a profound connection to Fisher scoring from Chapter 2, making it a natural bridge between classical statistics and modern machine learning.

At iteration t , LogitBoost maintains a model $F^{(t)}(x)$ in log-odds space. The predicted probability is:

$$p^{(t)}(x) = \frac{1}{1 + \exp(-F^{(t)}(x))} \quad (4.10)$$

The Working Response: LogitBoost constructs a pseudo-response (working response) for each training example:

$$z_i^{(t)} = \frac{y_i - p_i^{(t)}}{p_i^{(t)}(1 - p_i^{(t)})} \quad (4.11)$$

This is precisely the Newton-Raphson update from Chapter 2's treatment of GLMs. The denominator $p_i^{(t)}(1 - p_i^{(t)})$ is the Bernoulli variance, and the numerator is the residual. LogitBoost then fits a weak learner $h_t(x)$ to predict $z_i^{(t)}$ using weighted least squares with weights $w_i^{(t)} = p_i^{(t)}(1 - p_i^{(t)})$, making the connection between classical statistical estimation and modern boosting explicit.

LogitBoost: Same Algorithm, Different Estimator

The LogitBoost algorithm [17] uses identical formulas to logistic regression fit with iterative reweighed least squares (IRLS) [20], with one key difference: the choice of estimator.

Logistic Regression (IRLS) Hosmer & Lemeshow (2000), §4.4	LogitBoost Friedman, Hastie, Tibshirani (2000)
Working response: $z_i = \frac{y_i - \hat{\pi}_i}{\hat{\pi}_i(1 - \hat{\pi}_i)}$	Working response: $z_i = \frac{y_i - p(x_i)}{p(x_i)(1 - p(x_i))}$
Case weights: $v_i = \hat{\pi}_i(1 - \hat{\pi}_i)$	Weights: $w_i = p(x_i)(1 - p(x_i))$
Fit: Linear regression of z_i on x using weights v_i	Fit: Weighted least-squares regression of z_i to x_i using w_i
Estimator: Parametric (linear regression (with weights))	Estimator: Non-parametric (decision tree stumps)

Same formulation, same loss function, same iterative procedure—the only difference is the non-parametric estimator.

LogitBoost required base learners that could handle sample weights. Friedman's discovery was that *any* base learner could work if we fit to gradients instead of working responses. This generalization, replacing weighted least squares with functional gradient descent, unlocked the full potential of boosting for arbitrary loss functions.

Gradient Boosting Machines

Rather than deriving specialized boosting algorithms for every loss function, Friedman [18] introduced **gradient boosting machines**, a generic framework that works with any differentiable loss function. The key insight: fit weak learners to the *negative gradient* of the loss function, treating boosting as gradient descent in function space.

For a loss function $L(y, F(x))$, the negative gradient at iteration t is:

$$g_i^{(t)} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F^{(t-1)}} \quad (4.12)$$

The weak learner $h_t(x)$ is fit to predict these gradients $g_i^{(t)}$, and the model is updated:

$$F^{(t)}(x) = F^{(t-1)}(x) + \eta \cdot h_t(x) \quad (4.13)$$

where η is the learning rate (shrinkage parameter).

Newton Step: Adding Second-Order Information

Modern gradient boosting implementations improve upon first-order gradient boosting by incorporating second-order information, the Hessian. This is the same principle that made Fisher scoring faster than gradient descent in the credit score calibration example from Chapter 2. Fitted this way, gradient-boosted decision trees (GBDT) have an advantage over LogitBoost that *they do not require weighted fitting* [28].

For binary classification problems with log loss, the Hessian (second derivative) is:

$$h_i^{(t)} = \left. \frac{\partial^2 L(y_i, F(x_i))}{\partial F(x_i)^2} \right|_{F=F^{(t-1)}} = p_i^{(t)}(1 - p_i^{(t)}) \quad (4.14)$$

Modern algorithms use both gradient and Hessian to compute optimal leaf weights via a Newton step.

Leaf Weight Calculation: For a leaf ℓ containing training examples $\mathcal{I}_\ell$, the optimal weight is:

$$w_\ell = - \frac{\sum_{i \in \mathcal{I}_\ell} g_i}{\sum_{i \in \mathcal{I}_\ell} h_i + \lambda} \quad (4.15)$$

where λ is an L2 regularization parameter. The Newton-Raphson update is the negative gradient sum divided by the Hessian sum in a leaf node.

How Trees Make Updates

For categorical features with one-hot encoding, each category forms its own leaf. Unlike logistic regression, which produces log odds values in closed form, GBDT cannot solve for optimal leaf weights in one step, because the Hessian depends on current predictions.

At initialization with base score $F^{(0)} = \log \frac{\bar{y}}{1-\bar{y}}$, the leaf weight for category c becomes:

$$w_c = -\frac{\sum_{i:x_i=c}(p_0 - y_i)}{\sum_{i:x_i=c} p_0(1 - p_0)} = -\frac{p_0 - \bar{y}_c}{p_0(1 - p_0)} \quad (4.16)$$

where $\bar{y}_c$ is the mean target in category c and $p_0 = \bar{y}$ is the overall base probability.

The discrepancy arises because the Hessian uses the *base probability* p_0 (the overall mean) rather than the *category-specific probability* $\bar{y}_c$. At initialization, all predictions equal the base score, so:

$$h_i = p_0(1 - p_0) \quad \text{for all } i \quad (4.17)$$

If we instead used the category-specific Hessian $h_i^c = \bar{y}_c(1 - \bar{y}_c)$, the leaf weight would become:

$$w_c^* = -\frac{p_0 - \bar{y}_c}{\bar{y}_c(1 - \bar{y}_c)} \quad (4.18)$$

which converges to WOE as the base score approaches the category mean. In practice, with learning rate $\eta = 1.0$, GBDT converges to near-WOE values within 2-3 iterations as predictions become category-specific.

Gradient-boosted tree leaf weights are maximum-likelihood log-odds corrections for the region they cover. GBDT operates on exactly the same logic as Weight of Evidence (WOE) scorecards: both quantify how much the conditional odds shift relative to the base odds. This parallel reveals why gradient boosting has become so successful in credit scoring—it automates the same evidence-based reasoning that credit risk modelers have used for decades, but extends it to handle continuous

features, complex interactions, and high-dimensional data without requiring manual binning or WOE calculations.

Yet this power comes with a cost. While WOE scorecards make each feature’s contribution transparent, gradient boosting’s complex tree structures obscure how individual features influence predictions. A single feature may appear in multiple trees, at different depths, interacting with other features in ways that defy simple interpretation. This opacity became the central challenge for deploying tree ensembles in regulated domains like credit scoring, where model explainability is not optional but legally required.

The solution to this explainability problem would come from an unexpected source: a 1950s game theory paper about fairly dividing payoffs among cooperating players.

4.2 Game Theory and Machine Learning

The mathematical elegance of Turing’s weight of evidence finds its modern expression in SHAP (SHapley Additive exPlanations), revealing a profound connection between wartime codebreaking and contemporary machine learning interpretability.

Shapley values originated in cooperative game theory [30], where Lloyd Shapley addressed a fundamental question: how should players in a cooperative game fairly divide the total payout based on their contributions? The Shapley value is the weighted average of a player’s marginal contribution to all possible coalitions—the value the coalition gains by including that player.

This game-theoretic concept found new life in machine learning through Lundberg and Lee (2017) [24], who adapted Shapley values to interpret predictions. They coined the term **SHAP** (SHapley Additive exPlanations) to distinguish the machine learning application from the original game-theoretic method.

In predictive modeling, *features* play the role of players in the cooperative game, and *the prediction* represents the total payout to be fairly distributed among them. This framework extends Turing’s additive evidence structure from simple linear models to arbitrarily complex machine learning systems.

The Mathematical Foundation

The formal definition of Shapley values provides the mathematical bridge between cooperative game theory and machine learning interpretability. For a cooperative game with player set N and characteristic function v , the Shapley value $\phi_i(v)$ for player i is:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)] \quad (4.19)$$

where:

- S represents all possible coalitions not containing player i
- $v(S \cup \{i\}) - v(S)$ is the marginal contribution of player i to coalition S
- The coefficient $\frac{|S|!(|N|-|S|-1)!}{|N|!}$ weights each marginal contribution

This weighting scheme ensures that all possible orderings of players joining the coalition are considered equally, making the Shapley value the expected marginal contribution across all possible sequences.

From Coalitions to Features

When Lundberg and Lee adapted Shapley values for machine learning, they made a crucial conceptual leap. In the ML context:

- **Players** become **features** $x_1, x_2, \dots, x_p$
- **Coalitions** become **feature subsets** $S \subseteq \{1, 2, \dots, p\}$
- **Characteristic function** $v(S)$ becomes the **model prediction** for coalition S
- **Shapley values** become **SHAP values** explaining individual predictions

The SHAP value for feature j in instance x is:

$$\phi_j(x) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_x(S \cup \{j\}) - f_x(S)] \quad (4.20)$$

where F is the set of all features and $f_x(S)$ represents the expected model output when only features in set S are “present” and others are replaced by their expected values.

The Four Fundamental Axioms

Shapley proved that his value is the unique solution satisfying four desirable properties, which SHAP inherits:

1. Efficiency (Completeness) The sum of all SHAP values equals the difference between the prediction and the baseline:

$$\sum_{j=1}^p \phi_j(x) = f(x) - \mathbb{E}[f(X)] \quad (4.21)$$

2. Symmetry (Equal Treatment) If two features contribute equally to all possible coalitions, they receive equal SHAP values:

$$f_x(S \cup \{i\}) = f_x(S \cup \{j\}) \text{ for all } S \not\ni i, j \Rightarrow \phi_i(x) = \phi_j(x) \quad (4.22)$$

3. Dummy (Null Player) If a feature never changes the prediction regardless of which other features are present, its SHAP value is zero:

$$f_x(S \cup \{i\}) = f_x(S) \text{ for all } S \Rightarrow \phi_i(x) = 0 \quad (4.23)$$

4. Additivity (Linearity) For the sum of two games, SHAP values add:

$$\phi_i(f + g) = \phi_i(f) + \phi_i(g) \quad (4.24)$$

Why SHAP Succeeded Where Other Methods Failed

Previous explainability methods (LIME, permutation importance) lacked theoretical guarantees. SHAP succeeded in credit risk because:

- It produces the same explanations as traditional WOE scorecards for linear models (continuity with existing practice)
- Its axioms (efficiency, symmetry, dummy, additivity) map directly to regulatory requirements for fair lending
- TreeSHAP made it computationally feasible for production systems

This combination of theoretical rigor and practical performance was unprecedented in explainable AI.

This combination of axiomatic rigor and practical applicability becomes even more apparent when we examine SHAP's relationship to traditional credit scoring methods.

SHAP Values for Linear Models

For linear models without feature interactions, SHAP values have a closed-form analytical solution. Given a linear model:

$$f(x^{(i)}) = \beta_0 + \sum_{j=1}^p \beta_j x_j^{(i)} \quad (4.25)$$

the SHAP value for feature j in observation i is defined as:

$$\phi_j^{(i)} = \beta_j x_j^{(i)} - \mathbb{E}[\beta_j X_j] = \beta_j (x_j^{(i)} - \mathbb{E}[X_j]) \quad (4.26)$$

Replacing the expectation with the empirical mean:

$$\hat{\phi}_j^{(i)} = \beta_j \left(x_j^{(i)} - \frac{1}{n} \sum_{k=1}^n x_j^{(k)} \right) \quad (4.27)$$

The **efficiency axiom** ensures that the sum of feature contributions equals the deviation from the mean prediction:

$$\sum_{j=1}^p \phi_j^{(i)} = f(x^{(i)}) - \mathbb{E}[f(X)] \quad (4.28)$$

Hence, each prediction can be written as:

$$f(x^{(i)}) = \mathbb{E}[f(X)] + \sum_{j=1}^p \phi_j^{(i)} \quad (4.29)$$

SHAP Values as Centered Weights of Evidence

Turing's Naïve Bayes model expresses log-odds as an additive sum of logarithmic weights of evidence:

$$\log \frac{p(H_0 \mid s_1, \dots, s_p)}{p(H_1 \mid s_1, \dots, s_p)} = \sum_i w_i + \log \frac{p(H_0)}{p(H_1)} \quad (4.30)$$

Each weight of evidence:

$$w_i = \log \frac{p(s_i \mid H_0)}{p(s_i \mid H_1)} \quad (4.31)$$

represents the strength of evidence provided by feature s_i .

This formulation is mathematically equivalent to a **logistic regression** model:

$$\text{logit}(p_i) = \beta_0 + \sum_{j=1}^M \text{WOE}_j(x_{ij}) \quad (4.32)$$

where $\text{WOE}_j(x_{ij})$ is the evidence contributed by feature j , and $\beta_0 = \log \frac{p(H_0)}{p(H_1)}$ represents the prior log-odds.

The Connection to SHAP

The profound connection between Turing's weight of evidence and SHAP becomes clear when we consider how SHAP decomposes these models. For a logistic regression with weight of evidence transformations, the SHAP value for feature j is simply:

$$\phi_j = \beta_j \cdot [\text{WOE}_j(x_j) - \mathbb{E}[\text{WOE}_j]] \quad (4.33)$$

This reveals that SHAP values are **centered weights of evidence**: they measure how much each feature's evidence deviates from its expected contribution. Turing's

additive evidence framework thus provides the mathematical foundation for modern model interpretability.

In logistic models, SHAP values decompose the predicted log-odds into a baseline plus additive contributions:

$$\text{logit}(p_i) = \phi_0 + \sum_{j=1}^M \phi_{ij} \quad (4.34)$$

where

- $\phi_0 = \mathbb{E}[\text{logit}(p)]$ is the baseline (expected log-odds), and
- ϕ_{ij} is the Shapley contribution of feature j for observation i .

These can be interpreted as **centered weights of evidence**:

$$\phi_{ij} = \text{WOE}_j(x_{ij}) - \mathbb{E}[\text{WOE}_j] \quad (4.35)$$

Thus, the total deviation from the baseline equals the sum of centered evidences:

$$\Delta_i = \text{logit}(p_i) - \phi_0 = \sum_{j=1}^M \phi_{ij} \quad (4.36)$$

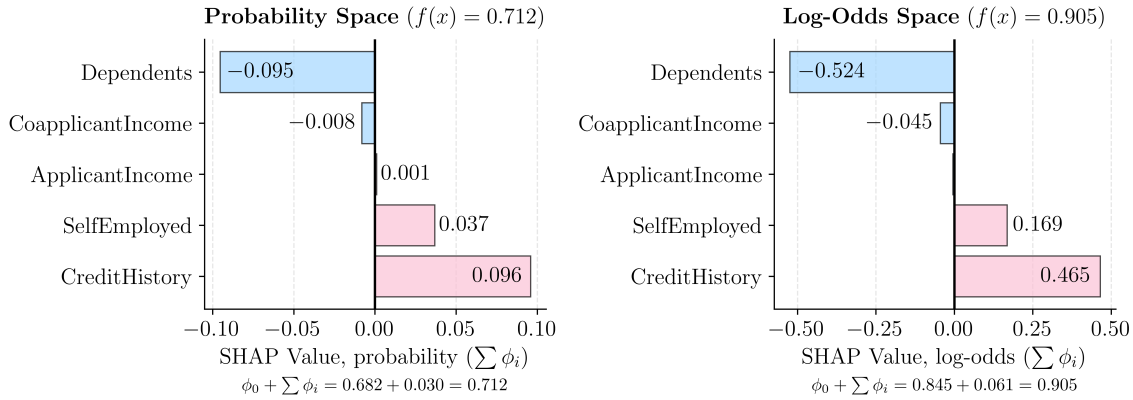


Figure 4.11: SHAP values computed in probability space versus log-odds space for a loan approval prediction. The left panel shows SHAP values in probability space, while the right panel shows the same values in log-odds space. Both panels demonstrate the efficiency axiom: the base value ϕ_0 plus the sum of feature contributions $\sum \phi_i$ equals the model's prediction $f(x)$.

Linear Equivalence

For linear or logistic models, SHAP values reproduce the additive WOE structure:

$$\phi_{ij} = \beta_j(x_{ij} - \mathbb{E}[x_j]) \quad (4.37)$$

This yields the equivalence:

$$\text{logit}(p_i) = \phi_0 + \sum_{j=1}^M \phi_{ij} \iff \text{logit}(p_i) = \log \frac{p(H_0)}{p(H_1)} + \sum_{j=1}^M \text{WOE}_j(x_{ij}) \quad (4.38)$$

where $\text{SHAP}_i = \text{WOE}_i - \mathbb{E}[\text{WOE}]$ and the SHAP baseline $\phi_0 = \mathbb{E}[\text{logit}(p)]$ represents the expected model output.

This establishes a direct continuity between Turing's additive *weights of evidence* and modern Shapley-based model explanations [24].

WOE and Linear SHAP Values

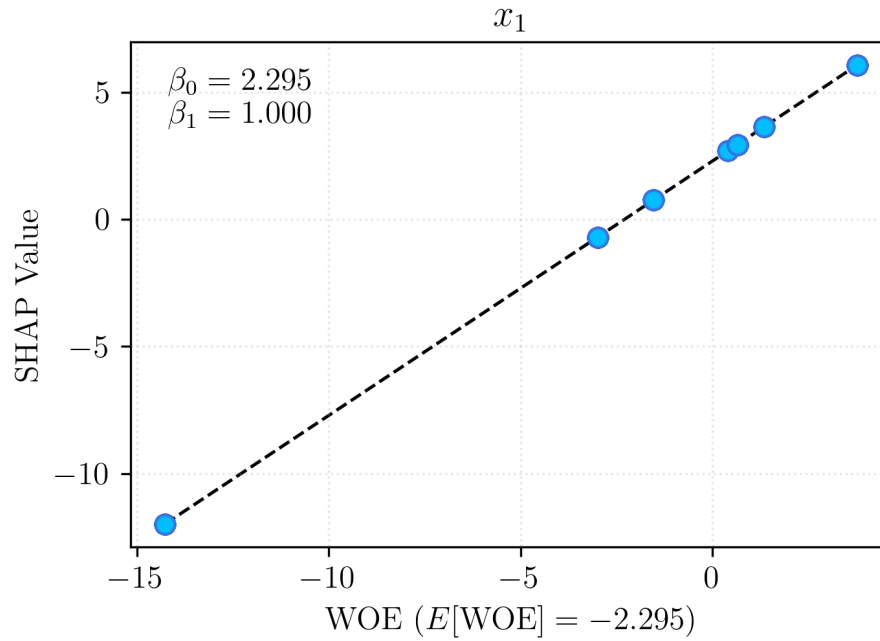


Figure 4.12: Linear relationship between WOE and SHAP values. The regression shows $\beta_1 \approx 1.0$ (perfect linear relationship) and $\beta_0 \approx -\mathbb{E}[\text{WOE}]$ (negative of the mean WOE), confirming that SHAP values are simply centered WOE values: $\text{SHAP} = \text{WOE} - \mathbb{E}[\text{WOE}]$.

4.3 SHAP Values for Tree Models

Tree ensembles solved the predictive accuracy problem that limited logistic regression and WOE scorecards. Random Forests and Gradient Boosting capture the complex nonlinearities and feature interactions that linear models cannot. But this success created a new problem: interpretability. A single decision tree is transparent, but an ensemble of hundreds of trees is a black box. Tracing how individual features influence predictions across the entire ensemble—each tree voting or contributing incrementally—becomes computationally intractable with naive approaches.

TreeSHAP [23] resolves this interpretability challenge. While the general SHAP computation requires evaluating 2^p coalitions (exponential complexity), TreeSHAP achieves a remarkable breakthrough for tree-based models by exploiting their fixed structure to compute exact SHAP values efficiently.

Why TreeSHAP Matters for Credit Risk

Unlike approximate methods like KernelSHAP that rely on sampling, TreeSHAP computes *exact* Shapley values by exploiting the tree's fixed decision paths. This exactness is critical for regulatory compliance: when a loan decision is challenged, TreeSHAP guarantees identical feature attributions every time—essential for adverse action notices and model documentation. Tree ensembles (Random Forests, XGBoost, LightGBM) have become the workhorses of modern credit scoring, outperforming logistic regression on nonlinear patterns. TreeSHAP provides principled, exact explanations for these powerful models without sacrificing their predictive accuracy.

The Tree Structure Advantage

TreeSHAP exploits a key property of tree models: their predetermined structure. Each prediction follows a fixed path defined by feature splits, and TreeSHAP traces this path to compute exact marginal contributions. Instead of evaluating all 2^p possible feature coalitions, TreeSHAP only considers features that appear on the decision path.

For ensembles, TreeSHAP computes SHAP values for each tree independently, then aggregates: Random Forests average across trees, while Gradient Boosting sums contributions scaled by the learning rate. This makes the algorithm both exact and efficient.

How TreeSHAP Works

TreeSHAP exploits the tree's fixed structure to avoid reasoning about all 2^p feature coalitions. For a given prediction, only features that appear along the instance's decision path can influence the output.

Crucially, TreeSHAP reasons over *unique path features*, not individual nodes. A feature like income may appear at multiple depths (e.g., “Income $\leq$ \$50k” at the root and “Income $\leq$ \$30k” deeper in the tree), but TreeSHAP treats it as a single feature. When evaluating a coalition that includes income, all income-based splits use the applicant’s actual income. When income is excluded from the coalition, those splits are averaged over the training data: if 60% of training applicants at that node went left, the algorithm weights the prediction as 60% from the left branch and 40% from the right branch.

The Algorithm:

- **Extract unique path features:** Identify which features were used to route instance $\mathbf{x}$ through the tree, collecting each feature only once even if it appears at multiple depths.
- **Evaluate feature coalitions:** For each subset of unique path features, compute the expected prediction. Features in the subset follow the instance’s actual values at all their splits; features outside are marginalized using training data branch proportions.
- **Compute Shapley values:** Apply the standard Shapley formula over unique path features. Each feature’s SHAP value measures its average marginal contribution across all possible coalitions.

For ensembles, TreeSHAP computes SHAP values for each tree independently, then aggregates: Random Forests average across trees, while Gradient Boosting sums contributions scaled by the learning rate.

TreeSHAP Example: Probability of Default Modeling

We illustrate TreeSHAP’s marginalization strategy with a decision tree for loan default prediction (features: income, debt ratio, time with bank). Consider an applicant with income = \$35k, debt_ratio = 0.3, following the path to leaf A ($P_{\text{default}}^A = 0.15$).

Empty Coalition: $f(\{\})$

When no features are known, TreeSHAP marginalizes over all branches using training data proportions (Figure 4.13):

$$\begin{aligned} f(\{\}) &= 0.6 \times (0.667 \times 0.15 + 0.333 \times 0.70) \\ &\quad + 0.4 \times (0.375 \times 0.50 + 0.625 \times 0.10) \\ &= 0.6 \times 0.333 + 0.4 \times 0.251 = 0.300 \end{aligned}$$

Coalition Evaluations

To compute SHAP values, we evaluate four coalition scenarios (Table 4.1):

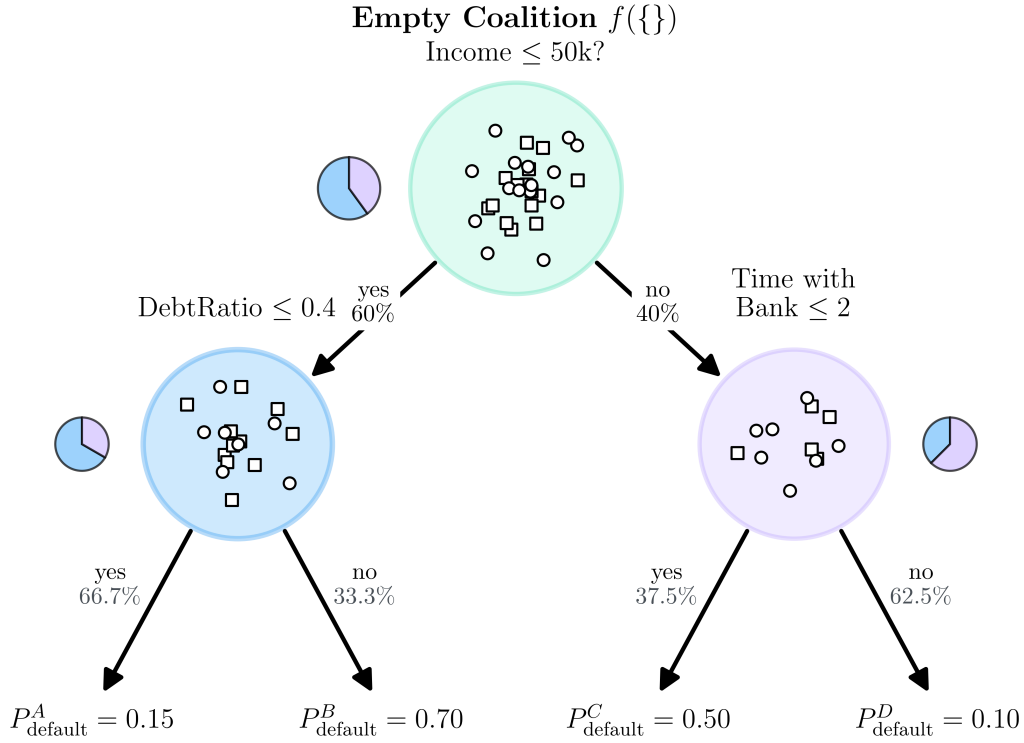


Figure 4.13: Empty coalition $f(\{\})$: TreeSHAP marginalizes over all branches using training data proportions. Pie charts show empirical splits: 60% income $\leq \$50k$, 40% income $> \$50k$. Yields $\mathbb{E}[f(X)] = 0.300$.

Table 4.1: Coalition evaluations

Coalition	Value
$f(\{\})$	0.300
$f(\{\text{income}\})$	0.333
$f(\{\text{debt_ratio}\})$	0.190
$f(\{\text{income}, \text{debt_ratio}\})$	0.150

Coalition $\{\text{income}\}$

When only income is known, it forces the left branch at the root; debt_ratio must be marginalized using training data proportions (Figure 4.14):

$$f(\{\text{income}\}) = 0.667 \times 0.15 + 0.333 \times 0.70 = 0.333$$

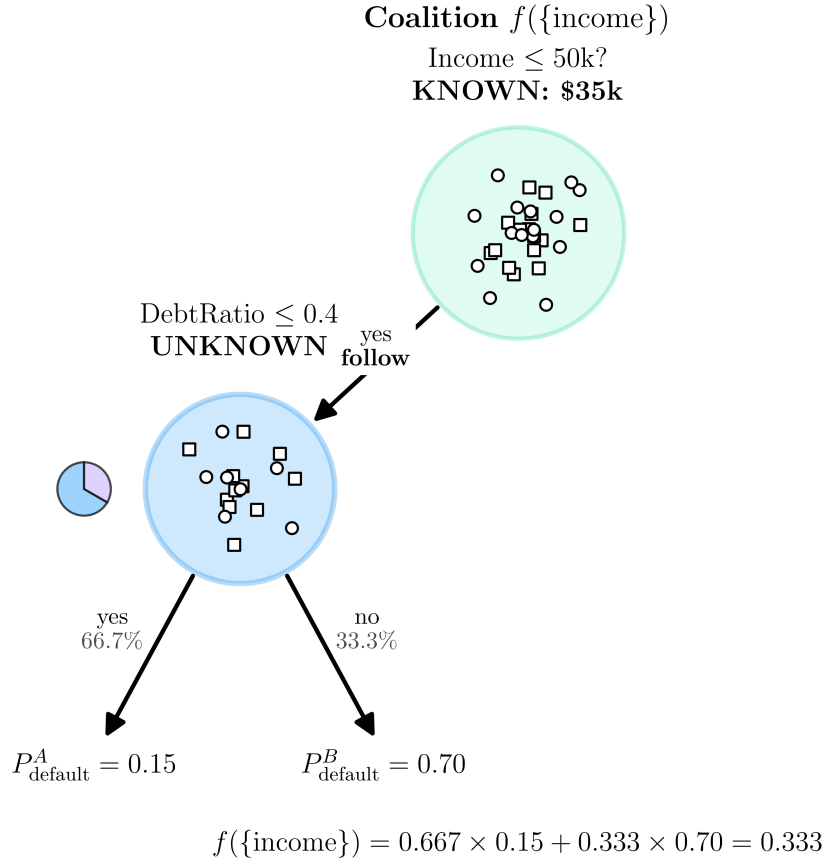


Figure 4.14: Coalition $f(\{\text{income}\})$: Income known, forcing left branch. Debt_ratio marginalized (66.7% to leaf A, 33.3% to leaf B).

Coalition $\{\text{debt_ratio}\}$

When only debt_ratio is known, income must be marginalized at the root (60% left, 40% right); debt_ratio = 0.3 then forces the left branch to leaf A (Figure 4.15):

$$f(\{\text{debt_ratio}\}) = 0.6 \times 0.15 + 0.4 \times (0.375 \times 0.50 + 0.625 \times 0.10) = 0.190$$

Coalition $\{\text{income}, \text{debt_ratio}\}$

When both features are known, we follow the deterministic path through the tree:

$$f(\{\text{income}, \text{debt_ratio}\}) = 0.15$$

Computing SHAP Values

Applying the Shapley formula $\varphi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} [f(S \cup \{j\}) - f(S)]$ where $|F| = 2$:

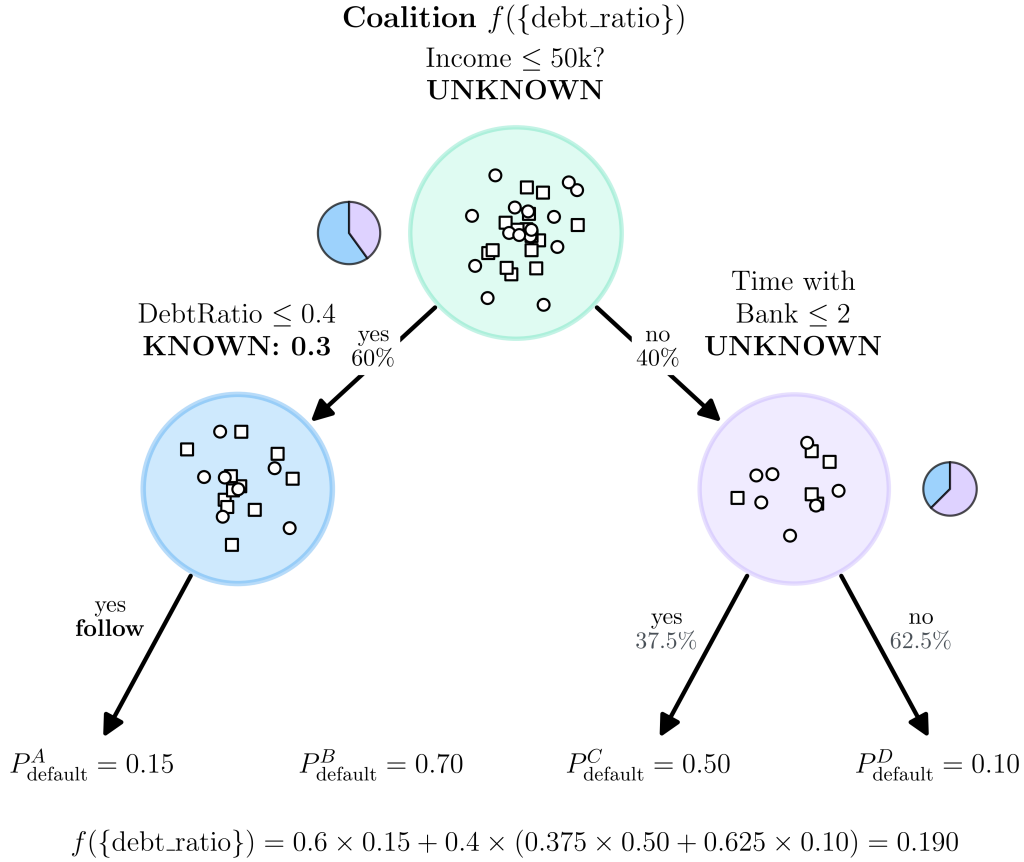


Figure 4.15: Coalition $f(\{\text{debt_ratio}\})$: Debt_ratio known, forcing left branch to leaf A. Income marginalized at root.

Income:

$$\begin{aligned}
 f(\{\text{income}\}) - f(\{\}) &= 0.333 - 0.300 = 0.033 \\
 w(0) &= \frac{0! \times (2 - 0 - 1)!}{2!} = \frac{1}{2} \\
 \frac{1}{2} \times 0.033 &= 0.0165 \\
 f(\{\text{income}, \text{debt_ratio}\}) - f(\{\text{debt_ratio}\}) &= 0.150 - 0.190 = -0.040 \\
 w(1) &= \frac{1! \times (2 - 1 - 1)!}{2!} = \frac{1}{2} \\
 \frac{1}{2} \times (-0.040) &= -0.020 \\
 \varphi_{\text{income}} &= 0.0165 - 0.020 = -0.004
 \end{aligned}$$

Debt ratio:

$$\begin{aligned}
 f(\{\text{debt_ratio}\}) - f(\{\}) &= 0.190 - 0.300 = -0.110 \\
 \frac{1}{2} \times (-0.110) &= -0.055 \\
 f(\{\text{income}, \text{debt_ratio}\}) - f(\{\text{income}\}) &= 0.150 - 0.333 = -0.183 \\
 \frac{1}{2} \times (-0.183) &= -0.092 \\
 \varphi_{\text{debt_ratio}} &= -0.055 - 0.092 = -0.147
 \end{aligned}$$

Since `time_with_bank` does not appear on this path, $\varphi_{\text{time_with_bank}} = 0$.

The SHAP values satisfy efficiency: $\varphi_{\text{income}} + \varphi_{\text{debt_ratio}} + \varphi_{\text{time_with_bank}} = -0.151 \approx 0.150 - 0.300$. Starting from a 30% base rate, income contributes -0.4 percentage points and debt ratio -14.7 percentage points, reaching the 15% prediction. TreeSHAP exploits the tree’s fixed structure, restricting computations to features on the decision path and marginalizing over training data when features are unknown.

4.4 SHAP Scorecards: Translating Complex Models into Point-Based Systems

Beyond model explanation, SHAP values enable an unexpected application in credit risk: building traditional scorecards from tree ensembles. While SHAP is primarily known for explaining individual predictions, its additive structure allows sophisticated models like XGBoost and Random Forests to be converted into the familiar scorecard format that credit risk managers use daily.

This works because SHAP values, like Weight of Evidence in logistic regression scorecards, represent additive contributions to log-odds. Each feature’s SHAP value can be scaled into scorecard points, preserving the model’s rank-ordering while providing the interpretable, auditable format required for regulatory compliance.

Converting SHAP Values to Points

The transformation from SHAP values to scorecard points preserves the additive structure that makes traditional scorecards interpretable:

$$\text{Score}_i = \text{Base Score} + \sum_{j=1}^p \text{Points}_{ij} \quad (4.39)$$

where Points_{ij} are derived from SHAP values through linear scaling:

$$\text{Points}_{ij} = \text{Scale Factor} \times \phi_{ij} \quad (4.40)$$

The scale factor is typically chosen to produce intuitive point ranges (e.g., 300-850 for FICO-style scores).

Mathematical Equivalence

For any model producing SHAP values, the scorecard maintains mathematical equivalence:

$$\text{logit}(p_i) = \phi_0 + \sum_{j=1}^p \phi_{ij} \quad (4.41)$$

$$\text{Score}_i = \text{Base} + \text{Factor} \times \sum_{j=1}^p \phi_{ij} \quad (4.42)$$

This ensures that scorecard rankings perfectly preserve model rankings while providing the familiar point-based format.

4.5 SHAP Distillation: Knowledge Transfer from Complex to Simple Models

SHAP scorecards convert model predictions into traditional formats while maintaining the underlying complexity. SHAP distillation takes a different approach: it creates a new, simpler model trained on SHAP values as features. The key challenge: SHAP values contain “baked-in” interactions from the complex model, requiring explicit interaction terms to preserve performance.

The Teacher-Student Framework

Teacher: A complex ensemble (e.g., CatBoost, XGBoost) with SHAP decomposition:

$$f(x_i) = \phi_0 + \sum_{j=1}^d \phi_{ij} \quad (4.43)$$

Student: A logistic regression trained on SHAP-derived features to approximate the teacher’s predictions.

Why Interactions Matter

SHAP values are additive, but they contain “baked-in” interactions from the teacher model. Each $\phi_j(x)$ already incorporates how feature j interacts with all other features. When we fit a simple linear model on SHAP values alone:

$$g(x) = \beta_0 + \sum_{j=1}^p \beta_j \phi_j(x) \quad (4.44)$$

we lose the ability to reconstruct these interactions. The student achieves only 70-80% of teacher performance.

To recover the teacher’s interaction structure, we must explicitly model interactions between SHAP values:

$$g(x) = \beta_0 + \sum_{j=1}^p \beta_j \phi_j(x) + \sum_{j < k} \gamma_{jk} \phi_j(x) \phi_k(x) \quad (4.45)$$

With interaction terms, performance retention increases to 85-95%.

Method 1: SHAP-Binned Distillation

Transform raw features into SHAP-informed categorical variables:

- Bin raw features x_j into discrete intervals
- For each bin, compute average SHAP contribution $\tilde{\phi}_{ij}$
- Train student logistic regression on binned SHAP features with interactions

Student Model:

$$\hat{p}_i = \sigma \left(\beta_0 + \sum_j \beta_j \tilde{\phi}_{ij} + \sum_{j < k} \beta_{jk} \tilde{\phi}_{ij} \tilde{\phi}_{ik} \right) \quad (4.46)$$

Method 2: SHAP-GAM Distillation

Use Generalized Additive Models to create smooth approximations:

- Fit GAM functions $g_j(x_{ij}) \approx \phi_{ij}$ for each feature
- Transform raw inputs through GAM predictions
- Include pairwise interaction terms
- Train student logistic regression on GAM-transformed features

Student Model:

$$\hat{p}_i = \sigma \left(\beta_0 + \sum_j \beta_j g_j(x_{ij}) + \sum_{j < k} \beta_{jk} g_j(x_{ij}) g_k(x_{ik}) \right) \quad (4.47)$$

4.6 Conclusion: Towards Cultural Convergence

The modern credit risk modeling workflow synthesizes what Breiman called “two cultures” into a unified approach:

1. **Build for accuracy:** Train gradient boosting models (XGBoost, LightGBM, CatBoost) that capture complex nonlinear patterns and feature interactions
2. **Explain individual decisions:** Use TreeSHAP to generate exact, reproducible feature attributions for adverse action notices and applicant inquiries

3. **Communicate at scale:** Distill SHAP values into traditional scorecards that credit officers can apply and regulators can audit
4. **Monitor and validate:** Track SHAP distributions over time to detect data drift, ensure stability, and verify that distilled models preserve the teacher’s decision logic

This workflow resolves the historical tension between algorithmic modeling and data modeling. Gradient boosting’s leaf weights represent maximum-likelihood log-odds adjustments—the algorithmic counterpart of Weight of Evidence scorecards. SHAP translates these adjustments into the additive feature contributions that regulators require and consumers can understand.

Practitioners gain the performance of ensemble methods without sacrificing transparency—models that satisfy both the data scientist’s demand for accuracy and the regulator’s requirement for interpretability. Yet questions remain: SHAP addresses individual explanations, but understanding global model behavior—feature importance rankings, interaction discovery, population-level patterns—requires additional tools. Research continues on SHAP’s relationship to causal inference, fair lending compliance, and the stability of explanations under model updates.

The trade-offs that defined credit modeling for decades—accuracy versus interpretability, prediction versus explanation, algorithms versus statistics—were not fundamental constraints, but artifacts of limited methodology. Under the right mathematical framework, these perspectives become complementary answers to the same question: how should evidence be combined, and how should credit be fairly assigned?

Breiman’s algorithmic culture and Fisher’s statistical rigor need not compete. From Fisher’s likelihood ratios through Turing’s sequential evidence to Shapley’s cooperative game theory, the thread connecting modern credit risk is consistent: decisions demand both predictive power and principled explanation. What once appeared to be a choice between scorecard transparency and ensemble performance has become a synthesis: algorithmic accuracy with statistical interpretability.

Bibliographical and Historical Notes

The foundational work on Classification and Regression Trees (CART) was published by Breiman, Friedman, Olshen, and Stone in 1984 [10]. The collaboration emerged from Breiman and Friedman’s consulting work in the 1970s, where they encountered problems that resisted traditional statistical methods. As documented in Stone’s recollections [31], the authors chose to publish as a monograph rather than journal articles, skeptical that statistical journals would accept work so focused on computation and algorithmic thinking.

Breiman’s intellectual trajectory from CART to Random Forests illustrates the evolution of ensemble methods. His 1996 paper on bagging [5] introduced bootstrap aggregation as a variance reduction technique, while his 2001 Random Forests paper [6] combined bagging with random feature selection. The original Random Forest implementation, developed by Breiman and Cutler in Fortran [9], used simple random sampling with replacement for feature selection at each split—the same feature could potentially be selected multiple times. The algorithm later evolved to use the Fisher-Yates shuffle for sampling without replacement, improving efficiency through $O(m)$ time complexity while maintaining the decorrelation properties essential to the method’s success. Liaw and Wiener’s 2002 R port [21] made Random Forests widely accessible to practitioners. Cutler’s remembrance [12] emphasizes Breiman’s willingness to challenge statistical orthodoxy and his focus on real-world problem-solving over theoretical elegance.

The tension between algorithmic and data modeling cultures reached its explicit expression in Breiman’s 2001 paper “Statistical Modeling: The Two Cultures” [8]. Writing in *Statistical Science*, Breiman argued that statistics had ceded ground to computer science by prioritizing parametric models over predictive accuracy. Friedman’s 2001 paper “Greedy Function Approximation: A Gradient Boosting Machine” [18] connected Breiman’s ensemble ideas to statistical optimization, showing that boosting minimizes loss functions through stagewise additive modeling. The LogitBoost paper [17] had earlier attempted to explain boosting through maximum likelihood, though Breiman remained unconvinced.

TreeSHAP, introduced by Lundberg and Lee in their 2020 paper “Local Explanations Using Unique Decision Paths” [25], solved the computational barrier to applying Shapley values to tree ensembles. Their algorithm exploits the tree’s predetermined structure to compute exact SHAP values in polynomial time, reducing complexity from $O(2^p)$ to $O(TLD^2)$. The 2018 consistency paper [22] established TreeSHAP’s theoretical properties, proving it satisfies local accuracy, missingness, and consistency—properties that other attribution methods lack.

The mathematical foundations trace to Lloyd Shapley’s 1953 work on cooperative game theory [30], which established axiomatic principles for fair allocation. Shapley’s framework, grounded in efficiency, symmetry, dummy player, and additivity axioms, formalized fairness as mathematics rather than intuition. The conceptual leap to machine learning occurred with Lundberg and Lee’s 2017 SHAP paper [24], which recast model features as players and predictions as payoffs. As documented in Molnar’s *Interpretable Machine Learning* [27], SHAP’s axiomatic foundation makes it essential for regulatory compliance in credit risk modeling.

The application of SHAP to credit risk represents a convergence of regulatory requirements and algorithmic innovation. SHAP scorecards convert ensemble

predictions into traditional formats while maintaining interpretability—a practical synthesis that would have satisfied both sides of Breiman’s two cultures debate.

References

- [1] J. H. Bennett. *Statistical Inference and Analysis: Selected Correspondence of R.A. Fisher*. Ed. by J. H. Bennett. Oxford: Oxford University Press, 1990.
- [2] Leo Breiman. ‘[Statistical Fraud Detection: A Review]: Comment’. In: *Statistical Science* 17.3 (2002), pp. 252–254.
- [3] Leo Breiman. ‘Additive Logistic Regression: A Statistical View of Boosting: Discussion’. In: *The Annals of Statistics* 28.2 (2000), pp. 374–377.
- [4] Leo Breiman. ‘Arcing classifiers’. In: *The Annals of Statistics* 26.3 (1998), pp. 801–824.
- [5] Leo Breiman. ‘Bagging predictors’. In: *Machine Learning* 24.2 (1996), pp. 123–140.
- [6] Leo Breiman. ‘Random forests’. In: *Machine Learning* 45.1 (2001), pp. 5–32.
- [7] Leo Breiman. ‘Reflections after refereeing papers for NIPS’. In: *The Mathematics of Generalization*. Ed. by David H. Wolpert. Reading, MA: Addison-Wesley, 1995, pp. 11–15.
- [8] Leo Breiman. ‘Statistical modeling: The two cultures’. In: *Statistical science* 16.3 (2001), pp. 199–231.
- [9] Leo Breiman and Adele Cutler. *Random Forests*. <https://www.stat.berkeley.edu/~breiman/RandomForests/>. Original Fortran and C code for Random Forests. 2001.
- [10] Leo Breiman et al. *Classification and Regression Trees*. Monterey, CA: Wadsworth & Brooks/Cole Advanced Books & Software, 1984. ISBN: 978-0412048418.
- [11] David R. Cox. ‘The regression analysis of binary sequences’. In: *Journal of the Royal Statistical Society Series B* 20.2 (1958), pp. 215–242.
- [12] Adele Cutler. ‘Remembering Leo Breiman’. In: *The Annals of Applied Statistics* 4.4 (Dec. 2010), pp. 1621–1633.
- [13] N. I. Fisher. ‘A Conversation with Jerry Friedman’. In: *Statistical Science* 30.2 (2015), pp. 268–295. DOI: 10.1214/14-STS509.
- [14] Ronald A. Fisher. ‘On the mathematical foundations of theoretical statistics’. In: *Philosophical Transactions of the Royal Society of London Series A* 222.594-604 (1922), pp. 309–368.
- [15] Ronald A. Fisher and Frank Yates. *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver and Boyd, 1938.

- [16] Yoav Freund and Robert E Schapire. ‘A decision-theoretic generalization of on-line learning and an application to boosting’. In: *Journal of computer and system sciences* 55.1 (1997), pp. 119–139.
- [17] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. ‘Additive logistic regression: a statistical view of boosting (with discussion and a rejoinder by the authors)’. In: *Annals of statistics* 28.2 (2000), pp. 337–407.
- [18] Jerome H Friedman. ‘Greedy function approximation: a gradient boosting machine’. In: *Annals of statistics* 29.5 (2001), pp. 1189–1232.
- [19] Jerome H Friedman. ‘The role of statistics in the data revolution?’ In: *International Statistical Review* 69.1 (2001), pp. 5–10.
- [20] David W. Hosmer and Stanley Lemeshow. *Applied Logistic Regression*. 2nd. New York: John Wiley & Sons, 2000.
- [21] Andy Liaw and Matthew Wiener. *Classification and Regression by randomForest*. 3. 2002, pp. 18–22. URL: <https://CRAN.R-project.org/doc/Rnews/>.
- [22] Scott M. Lundberg, Gabriel G. Erion, and Su-In Lee. ‘Consistent Individualized Feature Attribution for Tree Ensembles’. In: *arXiv preprint arXiv:1802.03888*. 2018.
- [23] Scott M. Lundberg, Gabriel G. Erion, and Su-In Lee. ‘Consistent Individualized Feature Attribution for Tree Ensembles’. In: *arXiv preprint arXiv:1802.03888* (2018). preprint. arXiv: 1802.03888 [cs.LG].
- [24] Scott M. Lundberg and Su-In Lee. ‘A unified approach to interpreting model predictions’. In: *Advances in Neural Information Processing Systems* 30 (2017), pp. 4765–4774.
- [25] Scott M. Lundberg et al. ‘From Local Explanations to Global Understanding with Explainable AI for Trees’. In: *Nature Machine Intelligence* 2.1 (2020), pp. 56–67.
- [26] Daniel McFadden. ‘Conditional logit analysis of qualitative choice behavior’. In: *Frontiers in Econometrics*. Ed. by P. Zarembka. Academic Press, 1973, pp. 105–142.
- [27] Christoph Molnar. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd. Comprehensive open-source guide to machine learning interpretability methods, including detailed analysis of SHAP advantages and limitations. Lulu.com, 2020. URL: <https://christophm.github.io/interpretable-ml-book/>.
- [28] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022. ISBN: 9780262046824.

- [29] John A. Nelder and Robert W. M. Wedderburn. ‘Generalized linear models’. In: *Journal of the Royal Statistical Society Series A* 135.3 (1972), pp. 370–384.
- [30] Lloyd S. Shapley. ‘A value for n-person games’. In: *Contributions to the Theory of Games* 2.28 (1953), pp. 307–317.
- [31] Charles J. Stone. ‘Selected Recollections of My Relationship with Leo Breiman’. In: *The Annals of Applied Statistics* 4.4 (Dec. 2010), pp. 1652–1655.